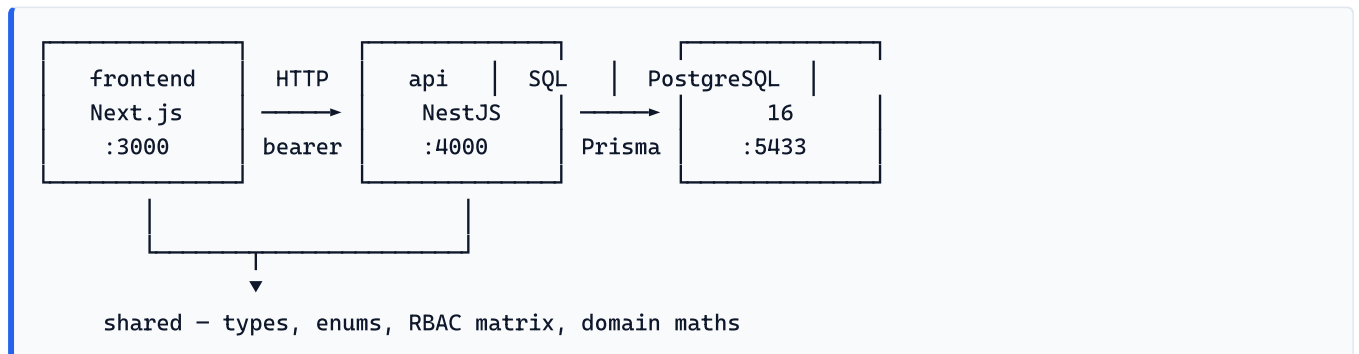


PeoplePay360 — HR & Payroll

An integrated HR and payroll platform. The employee record is the hub: contracts and working schedules supply the payroll context, attendance and time off capture what actually happened, salary structures and rules define how pay is computed, and a pay run turns eligible employees into validated payslips that can be downloaded as PDF or emailed.



The API is the only thing that touches the database, so authorisation cannot be sidestepped by calling a different client. The web app holds a session cookie and an HTTP client, nothing more.

Getting started

Requires **Node 20+** and **Docker**.

Everything in Docker

```
cp .env.example .env      # optional; every value has a working default
npm run docker:up         # builds and starts db + api + web
```

Open <http://localhost:3000>. That is the whole setup — the API applies its migrations and seeds the database on a fresh volume.

Running locally against a dockerised database

```
npm install
npm run setup      # start Postgres, build shared, migrate, seed
npm run dev        # API on :4000, web on :3000
```

Service	URL
Web	http://localhost:3000
API	http://localhost:4000/api
API reference (Swagger)	http://localhost:4000/api/docs
Component styleguide	http://localhost:3000/styleguide

Postgres is published on **5433**, not 5432, so it cannot collide with another Postgres already running on your machine.

Sign in

Every seeded account uses the password `password123`.

Email	Role	What it can reach
<code>admin@peoplepay360.com</code>	Admin	Everything, including user management
<code>payroll@peoplepay360.com</code>	HR Payroll Manager	All HR, plus payroll and salary configuration
<code>hr@peoplepay360.com</code>	HR Manager	All HR. No payroll at all
<code>employee@peoplepay360.com</code>	Employee	Only their own records

Each role lands on the first screen it can actually open, so the Employee and HR Manager accounts start at Employees rather than the overview.

Layout

people-466/	
api/	NestJS REST API – the only database client
prisma/	schema, migrations, seed
src/	
common/	guards, filters, decorators, validation, decimals
prisma/	database module
modules/	auth · employees · contracts · attendance
	time-off · payroll · config · dashboard
frontend/	Next.js 16 web client
src/	
app/	routes and server actions
components/	ui (library) · data (lists) · form (writes) · app (shell)
lib/	api client, session, access, fields, mutations, formatting
shared/	types, enums, RBAC matrix, pure domain maths
docker-compose.yml	db + api + web
docs/	see below

Documentation

Document	What it covers
docs/project-guide.md	Start here. The whole project in one read — stack, database, every folder, core functionality, and the reasoning behind each decision
docs/architecture.md	How the three packages fit, request flow, sessions, RBAC
docs/frontend.md	Component library, the form and mutation spine, adding a screen

Document	What it covers
docs/api.md	Every endpoint, and the conventions they share
docs/data-model.md	The entities, their relationships, and the payroll engine
docs/operations.md	Environment, Docker, migrations, deployment, troubleshooting

Commands

Command	Purpose
<code>npm run dev</code>	API and web together
<code>npm run dev:api</code> / <code>npm run dev:web</code>	One side only
<code>npm run build</code>	Build shared, then API, then web
<code>npm run typecheck</code>	Typecheck all three workspaces
<code>npm run lint</code>	Lint the web client
<code>npm run test</code>	API tests
<code>npm run db:migrate</code>	Create and apply a migration
<code>npm run db:seed</code>	Seed the database
<code>npm run db:reset</code>	Drop, re-migrate and reseed
<code>npm run db:studio</code>	Prisma Studio
<code>npm run docker:up</code> / <code>docker:down</code> / <code>docker:logs</code>	The full stack

Stack

NestJS 11 · Prisma 6 · PostgreSQL 16 · Next.js 16 · React 19 · TypeScript 5 · Tailwind CSS 4 · Animate UI + Radix · Motion · PDFKit · Azure Communication Services

Seed data

25 employees across 5 departments, 46 contracts including expired ones so period resolution is observable, roughly 1,650 attendance records over three months, 4 time off types with allocations and requests, 2 salary structures with 15 sequenced rules, and 2 completed pay runs so the dashboard has real history.

Some of it is deliberately imperfect, so the warning paths are demonstrable rather than theoretical: two employees have no bank details, several attendance records are missing a check-out or were manually corrected, and some contracts expire within 30 days.

What the rules actually are

These live in application logic rather than as hardcoded values. [docs/data-model.md](#) covers each in detail.

- **Contracts are period-scoped.** An employee may hold many over time; payroll resolves the one `RUNNING` contract whose dates overlap the period. Overlapping running contracts are rejected at save time.
- **Weekly hours are derived**, never accepted from a client — always from the day, start, end and break pattern, with overnight shifts rolling over correctly.
- **Leave balance is derived**, never stored: approved allocations minus approved requests. Approving a request links it to a specific allocation, so consumption is auditable.
- **Salary rules run in sequence**, each result entering scope under its code, so `NET` is literally `GROSS - PF - PT - TDS - ULD` rather than a hardcoded sum.
- **Payroll warnings block validation** — duplicate payslips, missing bank details, no applicable contract, negative net pay.
- **Money is `NUMERIC`**, never a float, so totals reconcile to the cent.

Security

- The JWT lives in an **httpOnly** cookie. Client-side JavaScript can never read it; the Next.js server attaches it as a bearer token when calling the API.
- Payslip PDFs go through a small Next.js proxy route, because a browser cannot set an `Authorization` header on a plain link. The proxy reads the cookie server-side.
- Every token is re-checked against the database on each request, so a deactivated account or a changed role takes effect immediately rather than at expiry.
- Client-side permission checks decide what is *offered*. The API re-checks every request and is the actual boundary.
- The API refuses to boot in production if `JWT_SECRET` is shorter than 32 characters.
- Request bodies carrying unknown fields are rejected outright rather than silently stripped.

Email

Payslip delivery goes through **Azure Communication Services** when `ACS_EMAIL_CONNECTION_STRING` and `ACS_SENDER_ADDRESS` are set, SMTP when only `SMTP_HOST` is, and otherwise nothing leaves — the attempt is recorded in the in-app outbox so the flow is demonstrable without credentials. The payslip PDF is attached either way, and every attempt lands in `/email-logs` with its status and, on failure, Azure's own message.

The sender must be a **verified** address on the ACS domain; Azure rejects any other username. See [docs/operations.md](#).

Roadmap

Approval chains with delegation; payroll journal export to accounting; employee document storage; multi-currency and multi-company; biometric or geofenced attendance; statutory report generation; and a scheduled job to auto-close contracts and expire allocations.